

Etapa 3 – Tabela de Resultados

Tamanho	Bubble Comp.	Bubble Trocas	Insertion Comp.	Insertion Mov.	Selection Comp.	Selection Trocas	Quick Comp.	Quick Mov.
10	45	17	17	17	45	7	22	22
20	190	104	104	104	190	18	62	62
1.000	499.500	243.577	243.577	243.577	499.500	994	10.732	10.732

Etapa 4 – Análise dos Resultados

a) Qual algoritmo realizou o menor número de comparações para 10 elementos?

O Insertion Sort, com 17 comparações — bem menos que Bubble e Selection (45 cada) e um pouco menos que o Quick Sort (22).

b) Qual algoritmo realizou menos trocas ou movimentações?

O Selection Sort, com apenas 7 trocas para 10 elementos.

c) O comportamento observado para 10 elementos permaneceu semelhante quando o tamanho aumentou para 20?

Sim, o padrão se manteve: Insertion Sort continuou com o menor número de comparações (104, empatado com Bubble Sort dessa vez) e Selection Sort continuou com o menor número de trocas (18).

d) O que aconteceu com a quantidade de operações quando o vetor passou para 1.000 elementos?

Todas as operações cresceram, mas de formas muito diferentes: Bubble e Selection Sort dispararam para 499.500 comparações, o Insertion Sort para 243.577, enquanto o Quick Sort ficou bem mais controlado, com apenas 10.732 comparações.

e) Bubble Sort, Insertion Sort e Selection Sort apresentam complexidade $O(n^2)$. Eles apresentaram exatamente a mesma quantidade de operações? Explique.

Não, apenas Bubble Sort e Selection Sort apresentaram exatamente o mesmo número de comparações (45, 190 e 499.500) — isso acontece porque os dois sempre percorrem todos os pares possíveis, independentemente da ordem dos dados. O Insertion Sort teve números de comparação diferentes (menores), porque seu laço interno (o `while`) pode parar antes de percorrer toda a parte ordenada, caso encontre logo um elemento menor que a chave. Já em relação às trocas/movimentações, os três tiveram valores bem diferentes: o Selection Sort faz no máximo uma troca por passagem (por isso teve muito menos trocas), enquanto Bubble e Insertion Sort tiveram exatamente o mesmo número de movimentações entre si

nesse experimento, pois ambos deslocam elementos individualmente a cada inversão encontrada.

f) Qual algoritmo apresentou maior crescimento no número de operações?

O Bubble Sort e o Selection Sort, empatados — ambos cresceram na mesma proporção quadrática ($O(n^2)$), saltando de 45 para 499.500 comparações (um crescimento de ~11.000 vezes) quando o tamanho do vetor aumentou 100 vezes.

g) Como o comportamento experimental do Quick Sort se diferenciou dos demais algoritmos?

O Quick Sort teve um crescimento muito mais controlado: enquanto os outros três algoritmos ultrapassaram as 240 mil operações com 1.000 elementos, o Quick Sort precisou de pouco mais de 10 mil — uma diferença de mais de 20 vezes. Isso confirma sua complexidade $O(n \log n)$, bem mais eficiente que a complexidade $O(n^2)$ dos outros três.

h) Os resultados encontrados são coerentes com as complexidades teóricas estudadas?

Sim. Bubble, Insertion e Selection Sort, todos $O(n^2)$, cresceram de forma muito mais agressiva que o Quick Sort, que é $O(n \log n)$ — exatamente como a teoria prevê. A proporção de crescimento (~11.000x para o Bubble Sort quando o tamanho aumentou 100x) também bate com o esperado matematicamente para uma complexidade quadrática ($100^2 = 10.000$).

i) Se você fosse responsável pelo sistema da central de distribuição e precisasse ordenar milhares de pedidos, qual dos quatro algoritmos escolheria?

Escolheria o Quick Sort, pois foi o único que manteve um número de operações significativamente menor conforme o volume de dados cresceu. Para milhares de pedidos (um cenário real de grande escala), a diferença de desempenho entre Quick Sort e os outros três algoritmos seria ainda mais crítica, tornando-o a opção mais eficiente e escalável para esse sistema.

Desafio Adicional

A organização inicial interfere na quantidade de operações de todos os algoritmos da mesma maneira?

Não, cada algoritmo reage de forma bem diferente:

Bubble Sort e Selection Sort: o número de comparações não muda (499.500 nos três casos), porque essas implementações sempre percorrem todos os pares possíveis, independente da ordem. O que muda são as trocas: zero no vetor já ordenado, máximo no invertido.

Insertion Sort: é o mais sensível à ordem dos dados, no sentido positivo. No vetor já ordenado, fez apenas 999 comparações e nenhuma movimentação — quase $O(n)$. No invertido, foi ao pior caso (499.500), precisando mover todo elemento para o início.

Quick Sort: é o mais sensível no sentido negativo. No vetor aleatório, se comportou muito bem (11.637 comparações). Mas nos vetores já ordenado e invertido, caiu no pior caso teórico (499.500 comparações — igual ao Bubble Sort), porque o pivô escolhido (o último elemento) deixa de dividir o problema de forma equilibrada. Essa implementação inclusive exigiu aumentar o limite de recursão do Python para não travar, evidenciando o quão desbalanceada fica a recursão nesses casos.

Conclusão: a organização inicial pode não afetar em nada (Bubble/Selection), favorecer enormemente (Insertion em dados ordenados) ou prejudicar drasticamente (Quick Sort em dados ordenados/invertidos) — não existe um padrão único.